

Appendix: Full $\mathfrak{so}(8)$ Closure Artifact Bundle

Steven Ettinger Jr
Independent Researcher
steven@disregardfiat.tech

May 14, 2026

Companion main manuscript. This appendix supports the AMS-style preprint *From Discrete Null-Lattice Growth to $\mathfrak{so}(8)$: Concrete Realization and Exact Lie Closure* (`papers/closure.tex` in this deposit). That manuscript states the growth-to- $\mathfrak{so}(8)$ chain and cites the exact rational certificate `artifacts/so8_symbolic_certificate.json`; this file inlines the generator script, JSON header, and the $\mathfrak{so}(3) + \Delta_4 \rightarrow \mathfrak{so}(4)$ toy pipeline for audit without opening separate tools.

Patch QFT and discrete numerics (reader contract). HQIV is formulated as a *patch theory*: quantum fields, actions, and physical readouts are attached to discrete patches—shell indices $m \in \mathbb{N}$, finite spacetime index types (e.g. `Fin 4`), and horizon-limited charts—rather than to a hypothetical fundamental smooth spacetime obtained as a mesh-refinement or ultraviolet limit. Accordingly, when this text refers to “QFT,” it means *patch QFT*: patching rules, finite measurement ledgers, and variational identities on the discrete spine; it is not the claim that the theory is *defined* by recovering ordinary continuum quantum field theory through such a limit. The numbers that admit the sharpest statements—mass and coupling ladders, curvature ratios, shell thermodynamics, and rational witnesses in `HQIV_LEAN`—are objects of *discrete mathematics* on $\mathbb{N}$ (combinatorial growth, cumulative simplex ledgers) together with the ordered field $\mathbb{R}$ as used in the proof assistant for analysis, bounds, and phenomenological display. Smooth-manifold or continuum field notation, when it appears, is a *translation layer* for comparison with standard physics literature; it does not supersede the discrete definitions.

Scope

This standalone appendix is intentionally minimal and contains only the symbolic proof-object pipeline and artifacts (exact rational certificates). It omits the full Lean file dump. The mirrored file tree for scripts, JSON, and Lean sources lives under `release/paper_refs_bundle_2026-05-06/` inside `companion-code.zip` from this Zenodo record (see `release/paper_refs_bundle_2026-05-06/MANIFEST.json`). After unpacking `companion-code.zip`, paths below are relative to `release/paper_refs_bundle_2026-05-06/`.

Remark (projected complex carrier). After the frozen complex structure J is fixed, coefficient-wise complexification promotes the octonion carrier to complex 8-tuples. In `HQIV_LEAN` this layer is Mathlib’s Hermitian model `EuclideanSpace  $\mathbb{C}$  (Fin 8)` (`WeakComplexOctonionCarrier` in `Hqiv/Algebra/WeakInComplexStructure.lean`), with `WithLp.ofLp/WithLp.toLp` mediating bare coefficient vectors `Fin 8  $\rightarrow$   $\mathbb{C}$` . The Hermitian pairing `weakCarrierCinner` is the

explicit slotwise sum, and `weakCarrierCinner_eq_inner` identifies it with Mathlib’s `inner` on `EuclideanSpace ℂ` (Fin 8) (so orthonormality lemmas apply after rewriting). The electroweak bookkeeping plane `weakJComplexDoublet` carries the standard Pauli half-generators; `Hqiv/Physics/WeakDoubletCarrierGaugeQuadratic.lean` packages the real 2×2 Gram matrix `weakWPlaneGramMatrixgv` for the (σ^1, σ^2) directions on the VEV ray. At the outer-horizon gauge data $(g, v) = (\text{su2CouplingDerived}, \text{vacuumExpectationValueGauge})$ it is a scalar multiple of the identity, hence both eigenvalues equal `weakWPlaneGramEigenvaluegv`; in particular `weakWPlaneGramMatrix_trace` gives $\text{trace} = 2\lambda$ and `weakWPlaneGramMatrix_det` gives $\det = \lambda^2$ for that common eigenvalue λ . The bridge `M_W_derived^2 = 8 * weakWPlaneGramEigenvalue...` matches the factor-8 packaging used in the main manuscript, and `ew_carrier_gram_mass_certificate` chains this spectral data directly to the rational witnesses `boson_witness_M_W / boson_witness_m_H` in `Hqiv/Physics/DerivedGaugeAndLeptonSector.lean`. The same file adds a carrier quartic `higgsQuarticPotentialCarrier` built from `higgsCarrierNormSq` (the squared carrier norm) and the geometric gauge vev anchor, with `m_H_sq_eq_two_lambda_times_vgauge_sq` implementing the $m_H^2 \propto \lambda v_{\text{gauge}}^2$ normalization with `higgsQuarticLambdaGaugeWitness` defined from existing masses (no new tunable parameter). Triality generation tags `So8RepIndex` connect to the harmonic ladder through `weakPhiOfShellOnSo8Rep` and the closed-form α^{-1} readout `weakOneOverAlphaEMatRep` (aligned with `one_over_alpha_EM_derived` in `Hqiv/Physics/SM_GR_Unification.lean`). For machine-readable PDG comparisons, rational `#eval` mirrors and small example bounds appear at the end of `Hqiv/Physics/WeakDoubletCarrierGaugeQuadratic.lean`. The narrative anchor in the long-form paper is the “Projected complex carrier” paragraph in `papers/paper/octonion_lightcone_to_oshoracle.tex`; the electroweak witness ladder is summarized in `papers/hqiv_lean_from_combinatorics_to_mass_spectrum.tex`.

Current Lean build split

The repository exposes the following *named* Lake targets (see `lakefile.toml` for the authoritative list):

- `lake build HQIVPaperClaims`: lightweight target for `papers/closure.tex` using `Hqiv/Geometry/OctonionicLightCone.lean` and `Hqiv/SO8ClosureSymbolic.lean` only (no satisfiability formalism in this mirror; no heavy matrix-certificate elaboration).
- `lake build HQIVSO8Closure`: heavyweight matrix-closure target (`Hqiv/GeneratorsLieClosureData*.lean`, `Hqiv/GeneratorsLieClosure.lean`, `Hqiv/LieBracketCell.*`, `Hqiv/Algebra/SO8ClosureAbstract.lean`). Lie-bracket coefficients are elaborated module-by-module; the linear-independence step in `Hqiv/GeneratorsLieClosure.lean` routes through the single Gram-matrix axiom `so8CoordMatrix_transpose_mul_self` in `Hqiv/So8CoordMatrix.lean` (documented bridge to the numeric orthonormality script). Expect a long cold-cache build. *RAM*: default parallel Lake builds of the 784 `LieBracketCell` modules can exceed $\sim 100\text{GB}$; use a full repository checkout and `scripts/build_hqiv_so8_closure_lowmem.sh` (`LEAN_NUM_THREADS=1`, `lake build ... -j 1`). The frozen `paper\_refs\_bundle\_2026-05-06/` mirror omits those `.lean` sources and supports `lake build HQIVPaperClaims` only; see `Hqiv/LieBracketCell/README.md` inside the bundle.
- `lake build HQIVStrongColorSu3Certificate`: optional `su(3)` color chart certificate (auto-generated `@[simp]` atoms for nonzero f^{abc} in `Hqiv/Physics/`

StrongColorSu3fStructureSimp.lean; not part of the default HQIVLEAN glob).

- Hqiv/Algebra/WeakFromLeftMulOctonion.lean: electroweak-style *definitions* from $L(e_i)$ (complex-structure J , candidate $W^\pm$, Z) with proved skewness only; $\mathfrak{su}(2)$ closure for weak isospin remains in Hqiv/Algebra/SMEmbedding.lean.

The $\mathrm{SO}(8)$ paper targets and their referenced files are included in `release/paper_refs_bundle_2026-05-06/`; the $\mathrm{SU}(3)$ certificate target is built in-repo on demand (and in CI) but is not duplicated in this frozen bundle.

Exact Symbolic Certificate (New)

Generator script

Unpack `companion-code.zip` if needed, then `cd` into `release/paper_refs_bundle_2026-05-06/`, then run:

```
python3 scripts/generate_symbolic_lie_closure_certificate.py
```

This script computes a rank-28 closure basis and all bracket coefficients exactly as rational numbers, and writes:

```
artifacts/so8_symbolic_certificate.json
```

`scripts/generate_symbolic_lie_closure_certificate.py`

```
#!/usr/bin/env python3
"""
Generate an exact (symbolic/rational) Lie-closure certificate for so(8)
from the HQIV seed generators  $\mathfrak{g}_2$  union  $\{\Delta\}$ , without modifying existing
floating-point certificate files.

Output:
- JSON certificate with:
  * exact basis (packed 28-vectors over  $\mathbb{Q}$ )
  * exact structure coefficients  $c_{\{ij\}}^k$  over  $\mathbb{Q}$ 
  * denominator statistics

Usage:
cd ~/Repos/HQIV_LEAN
PYTHONPATH=~/Repos/HQIV python3 scripts/generate_symbolic_lie_closure_certificate.py
PYTHONPATH=~/Repos/HQIV python3 scripts/generate_symbolic_lie_closure_certificate.py \
  --output artifacts/so8_symbolic_certificate.json

By default the script prints the 14 concrete  $\mathfrak{g}_2$  seed matrices and  $\Delta$ 
(8x8, exact integers after rounding floats from HQVM) to stdout for manuscript / appendix
listings. Use --no-print-matrices to suppress.
"""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path
from typing import Iterable

import sympy as sp

_REPO_LEAN = Path(__file__).resolve().parent.parent
_REPO_HQIV = _REPO_LEAN.parent / "HQIV"
```

```

if _REPO_HQIV.exists():
    sys.path.insert(0, str(_REPO_HQIV))

def _to_sympy_matrix(m) -> sp.Matrix:
    """Convert numpy-like matrix with numeric entries to exact sympy Matrix."""
    rows = []
    for i in range(8):
        row = []
        for j in range(8):
            # Current HQVM seed entries are integral / sign values.
            row.append(sp.Rational(int(round(float(m[i][j]))), 1))
        rows.append(row)
    return sp.Matrix(rows)

def _pack_antisym(m: sp.Matrix) -> sp.Matrix:
    """Pack 8x8 antisymmetric matrix to 28x1 vector (upper triangle i<j)."""
    return sp.Matrix([m[i, j] for i in range(8) for j in range(i + 1, 8)])

def _is_zero_matrix(m: sp.Matrix) -> bool:
    return all(v == 0 for v in m)

def _commutator(a: sp.Matrix, b: sp.Matrix) -> sp.Matrix:
    return a * b - b * a

def _rank_of_columns(cols: Iterable[sp.Matrix]) -> int:
    cols = list(cols)
    if not cols:
        return 0
    return sp.Matrix.hstack(*cols).rank()

def load_seed_g2_and_delta() -> tuple[list[sp.Matrix], sp.Matrix]:
    """Return exact SymPy 8x8 matrices for the 14-element ‘g2’ basis and ‘Delta’."""
    from HQVM.matrices import OctonionHQIVAlgebra

    alg = OctonionHQIVAlgebra(verbose=False)
    g2 = [_to_sympy_matrix(g) for g in alg.g2_basis]
    delta = _to_sympy_matrix(alg.Delta)
    return g2, delta

def format_matrix_8x8(m: sp.Matrix, *, col_width: int = 4) -> str:
    """Human-readable 8x8 block (entries as exact rationals / integers)."""
    lines = []
    for i in range(8):
        cells = [str(sp.Rational(m[i, j])).rjust(col_width) for j in range(8)]
        lines.append(" ".join(cells))
    return "\n".join(lines)

def print_proof_seed_matrices(
    g2: list[sp.Matrix],
    delta: sp.Matrix,
    *,
    stream=sys.stdout,
) -> None:
    """Print the concrete matrices cited in the Lie-closure proof (seed before bracket closure)."""
    stream.write(
        "\n# === Proof seed: 14 ×  $\frac{g}{2}$  generators +  $\Delta$  (8×8 each) ===\n"
    )
    for i, mat in enumerate(g2):
        stream.write(f"\n## g2_basis[{i}]\n")
        stream.write(format_matrix_8x8(mat))

```

```

        stream.write("\n")
        stream.write("\n## Delta (phase-lift generator on the chosen plane)\n")
        stream.write(format_matrix_8x8(delta))
        stream.write("\n")

def build_exact_basis(max_iter: int = 80, seed: list[sp.Matrix] | None = None) -> list[sp.Matrix]:
    """Build a rank-28 Lie-closure basis over Q from g2 union {Delta}.

    If 'seed' is omitted, matrices are loaded from 'OctonionHQIValgebra' once.
    """
    if seed is None:
        g2, delta = load_seed_g2_and_delta()
        seed = g2 + [delta]
    basis: list[sp.Matrix] = []
    basis_cols: list[sp.Matrix] = []

    # Keep seed order, add only rank-increasing directions.
    for m in seed:
        col = _pack_antisym(m)
        if _rank_of_columns(basis_cols + [col]) > len(basis_cols):
            basis.append(m)
            basis_cols.append(col)

    if len(basis_cols) == 28:
        return basis

    # Lie-closure growth by commutators, exact rank tests over Q.
    for _ in range(max_iter):
        grew = False
        n = len(basis)
        for i in range(n):
            for j in range(i + 1, n):
                c = _commutator(basis[i], basis[j])
                if _is_zero_matrix(c):
                    continue
                col = _pack_antisym(c)
                if _rank_of_columns(basis_cols + [col]) > len(basis_cols):
                    basis.append(c)
                    basis_cols.append(col)
                    grew = True
                if len(basis_cols) == 28:
                    return basis

        if not grew:
            break

    raise RuntimeError(f"Failed to reach rank 28; got rank {len(basis_cols)}")

def coeff_tensor_exact(basis: list[sp.Matrix]) -> list[list[list[sp.Rational]]]:
    """Compute exact structure constants  $c_{ij}^k$  in the chosen basis."""
    cols = [_pack_antisym(m) for m in basis]
    B = sp.Matrix.hstack(*cols) # 28x28
    if B.rank() != 28:
        raise RuntimeError("Basis matrix is not full rank.")

    coeff: list[list[list[sp.Rational]]] = []
    for i in range(28):
        row = []
        for j in range(28):
            br = _commutator(basis[i], basis[j])
            v = _pack_antisym(br)
            c = B.LUsolve(v) # exact rational vector
            # Ensure exact identity in packed coordinates.
            if B * c != v:
                raise RuntimeError(f"Symbolic solve verification failed at ({i}, {j}).")
            row.append([sp.Rational(c[k]) for k in range(28)])
        coeff.append(row)

```

```

    return coeff

def _rat_to_pair(q: sp.Rational) -> tuple[int, int]:
    return int(q.p), int(q.q)

def _matrix_to_pairs(m: sp.Matrix) -> list[list[list[int]]]:
    out = []
    for i in range(m.rows):
        row = []
        for j in range(m.cols):
            p, q = _rat_to_pair(sp.Rational(m[i, j]))
            row.append([p, q])
        out.append(row)
    return out

def main() -> None:
    parser = argparse.ArgumentParser(description="Generate exact symbolic so(8) closure certificate.")
    parser.add_argument(
        "--output",
        default=str(_REPO_LEAN / "artifacts" / "so8_symbolic_certificate.json"),
        help="Output JSON path (default: artifacts/so8_symbolic_certificate.json)",
    )
    parser.add_argument("--max-iter", type=int, default=80, help="Max closure growth iterations.")
    parser.add_argument(
        "--print-matrices",
        action=argparse.BooleanOptionalAction,
        default=True,
        help="Print 14 g2 + Delta 8x8 matrices to stdout (default: on).",
    )
    parser.add_argument(
        "--print-full-basis",
        action="store_true",
        help="After closure, also print all 28 basis 8x8 matrices (large).",
    )
    args = parser.parse_args()

    out_path = Path(args.output)
    out_path.parent.mkdir(parents=True, exist_ok=True)

    g2, delta = load_seed_g2_and_delta()
    if args.print_matrices:
        print_proof_seed_matrices(g2, delta)

    basis = build_exact_basis(max_iter=args.max_iter, seed=g2 + [delta])
    if args.print_full_basis:
        print("\n# === Full Lie-closure basis (28 matrices) ===\n", file=sys.stdout)
        for k, mat in enumerate(basis):
            print(f"\n## closure_basis[{k}]\n{format_matrix_8x8(mat)}\n", file=sys.stdout)
    coeff = coeff_tensor_exact(basis)

    denoms = [int(c.q) for i in range(28) for j in range(28) for c in coeff[i][j]]
    max_denom = max(denoms) if denoms else 1

    payload = {
        "description": "Exact symbolic Lie-closure certificate over Q for HQIV g2 + Delta in so(8)",
        "basis_count": len(basis),
        "basis_packed_q": [_matrix_to_pairs(_pack_antisym(m)) for m in basis],
        "coeff_q": [
            [list(_rat_to_pair(c)) for c in coeff[i][j]] for j in range(28)]
            for i in range(28)
        ],
        "stats": {
            "max_denominator": max_denom,
            "nonzero_coeff_count": sum(
                1 for i in range(28) for j in range(28) for c in coeff[i][j] if c != 0
            )
        }
    }

```

```

        ),
    },
}

with open(out_path, "w", encoding="utf-8") as f:
    json.dump(payload, f, indent=2)

print(f"Wrote symbolic certificate: {out_path}")
print(f"Basis size: {len(basis)} (expected 28)")
print(f"Max coefficient denominator: {max_denom}")

if __name__ == "__main__":
    main()

```

artifacts/so8_symbolic_certificate.json (header excerpt)

```

{
  "description": "Exact symbolic Lie-closure certificate over Q for HQIV g2 + Delta in so(8)",
  "basis_count": 28,
  "basis_packed_q": [
    [
      [
        0,
        1
      ]
    ],
    [
      [
        0,
        1
      ]
    ],
    [
      [
        0,
        1
      ]
    ],
    [
      [
        0,
        1
      ]
    ],
    [
      [
        0,
        1
      ]
    ],
    [
      [
        0,
        1
      ]
    ],
    [
      [
        0,
        1
      ]
    ],
    [
      [
        0,
        1
      ]
    ],
    [
      [
        0,
        1
      ]
    ],
    [
      [
        -2,
        1
      ]
    ],
    [

```



```
]
],
[
```

Toy Model Certificate: $\mathfrak{so}(3) + \Delta_4 \rightarrow \mathfrak{so}(4)$

This compact pedagogical artifact mirrors the exact-rational certificate pipeline in the smallest nontrivial closure setting.

scripts/generate_symbolic_so4_certificate.py

```
#!/usr/bin/env python3
"""
Generate an exact rational toy-model certificate for so(4) closure from
so(3) + Delta4, where Delta4 = J14.

Output:
  artifacts/so4_symbolic_certificate.json
"""

from __future__ import annotations

import json
from pathlib import Path

import sympy as sp

def j(a: int, b: int, n: int = 4) -> sp.Matrix:
    """Standard antisymmetric generator J_ab (1-indexed)."""
    m = sp.zeros(n, n)
    m[a - 1, b - 1] = 1
    m[b - 1, a - 1] = -1
    return m

def pack_antisym(m: sp.Matrix) -> sp.Matrix:
    return sp.Matrix([m[i, k] for i in range(4) for k in range(i + 1, 4)])

def rat_pair(x: sp.Rational) -> list[int]:
    q = sp.Rational(x)
    return [int(q.p), int(q.q)]

def main() -> None:
    out = Path(__file__).resolve().parent.parent / "artifacts" / "so4_symbolic_certificate.json"
    out.parent.mkdir(parents=True, exist_ok=True)

    basis = [j(1, 2), j(1, 3), j(1, 4), j(2, 3), j(2, 4), j(3, 4)]
    basis_names = ["J12", "J13", "J14", "J23", "J24", "J34"]
    B = sp.Matrix.hstack(*[pack_antisym(x) for x in basis]) # 6x6

    # Seed: so(3) on first 3 coords plus Delta4 = J14
    seed = [j(1, 2), j(1, 3), j(2, 3), j(1, 4)]
    seed_rank = sp.Matrix.hstack(*[pack_antisym(x) for x in seed]).rank()

    coeff = []
    for i in range(6):
        row = []
        for k in range(6):
            br = basis[i] * basis[k] - basis[k] * basis[i]
            v = pack_antisym(br)
```



```

    0,
    1
  ],
  [
    0,
    1
  ],
  [
    0,
    1
  ]
],
[
  [
    0,
    1
  ],
  [
    0,
    1
  ],
  [
    0,
    1
  ],
  [
    -1,
    1
  ],
  [
    0,
    1
  ],
  [
    0,
    1
  ]
],
[
  [
    0,
    1
  ],
  [
    0,
    1
  ],
  [
    0,
    1
  ],
  [
    0,
    1
  ],
  [
    -1,
    1
  ],
  [
    0,
    1
  ]
],
[
  [
    0,
    1

```

],	
[	
1,	
1	
],	
[	
0,	
1	
],	
[	
0,	
1	
],	
[	